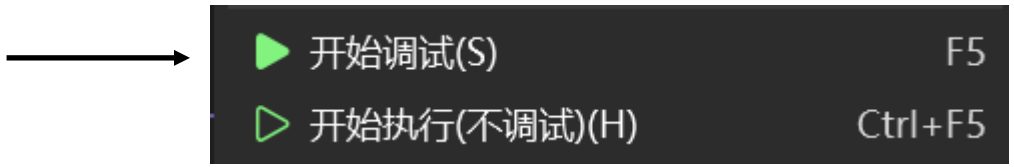
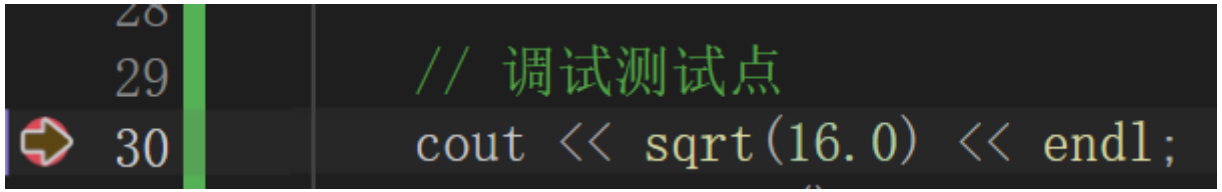
1.1如何开始调试?如何结束调试?



点击红色停止按钮 / Shift+F5结束调试

VS2026中点击“本地Windows调试器”按钮 / 按F5启动调试

1.2/1.3/1.4



如何在一个函数中每个语句单步执行?

在碰到cout/sqrt等系统类/系统函数时，如何一步完成这些系统类/系统函数的执行而不1.3要进入到这些系统类/函数的内部单步执行?

如果已经进入到cout/sqrt等系统类/系统函数的内部，如何跳出并返回自己的函数?

1.5/1.6

```
34    arrayFunc(arr1D, 3);          // 自定义函数（知识点1.5/1.6/3.6）
```

•按F10：一步执行完函数（不进入内部）

```
37    arrayFunc(arr1D, 3);
38
39
40    return 0;    已用时间 <= 1ms
```

在碰到自定义函数的调用语句(例如在main中调用自定义的fun函数)时，如何一步完成自定义函数的执行而不要进入到这些自定义函数的内部单步执行?

•按F11：进入函数内部单步执行

```
14    void arrayFunc(int* arr, int len) {
15        for (int i = 0; i < len; i++) {
16            arr[i] = arr[i] * 2;
17        }
18    }
```

在碰到自定义函数的调用语句(例如在main中调用自定义的fun函数)时，如何转到被调用函数中单步执行?

2.1查看形参/自动变量的变化情况

```
14    void arrayFunc(int* arr, int len) {
15        for (int i = 0; i < len; i++) {
16            arr[i] = arr[i] * 2;
17        }
18    }
```

```
arrayFunc(arr1D, 3);
```

i自动变量，len形参

名称	值	类型
> arr	0x000000c8dab2fa18 {1}	int *
i	200	int
len	3	int

2.2查看静态局部变量的变化情况(该静态局部变量所在的函数体内/函数体外)

```
8     void staticVarDemo() {
9         static int staticLocal = 0;  // 静态局部变量
10        staticLocal++;
11        cout << "staticLocal = " << staticLocal << endl;
12    }
```

名称	值	类型
staticLocal	1	int

内部

```
31        staticVarDemo();
```

名称	值	类型
sqrt	0x00007ff7a0626276 {vs2026test.exe!sqrt}	void *

外部

文件1

```cpp
static int staticGlobalA = 100;  // 静态全局变量（仅本文件可见）
int globalVar = 200;             // 外部全局变量
void showOther();
void testStaticGlobal() {   已用时间 <= 1ms
    cout << "DebugTest.cpp 中的 staticGlobalA = " << staticGlobalA << endl;
}

void testExternGlobal() {
    cout << "globalVar = " << globalVar << endl;
}

int main() {
    testStaticGlobal();
    testExternGlobal();
    showOther();
    return 0;
```

文件2

```cpp
#include <iostream>
using namespace std;

static int staticGlobalA = 999;  // 同名静态全局变量（仅本文件可见）
extern int globalVar;            // 声明外部全局变量

void showOther() {
    cout << "other.cpp 中的 staticGlobalA = " << staticGlobalA << endl;
    cout << "外部 globalVar = " << globalVar << endl;
}
```

2.3/2.4的调试程序

•静态全局变量虽然变量名相同，但作用域仅限于各自文件，内存地址不同，互不影响

testStaticGlobal内部（文件一）

名称	值	类型
globalVar	200	int
staticGlobalA	100	int

Showother内部（文件2）

名称	值	类型
globalVar	200	int
staticGlobalA	999	int

3.1 char/int/float等简单变量

```
21    int a = 10;
22    char b = 'B';
23    float c = 0.123F;
24    int* p = &a;
```

a	10	int
b	66 'B'	char
c	0.123000003	float

3.2 指向简单变量的指针变量(如何查看地址、值?)

3.2/3.3/3.4的调试代码

```
int* p = &a;                    // 指向简单变量的指针
int arr1D[3] = { 1, 2, 3 }; // 一维数组
int(*pArr)[3] = &arr1D;    // 指向一维数组的指针
```

p	0x000000d4f0cffb94 {10}	int *
	10	int

值

地址

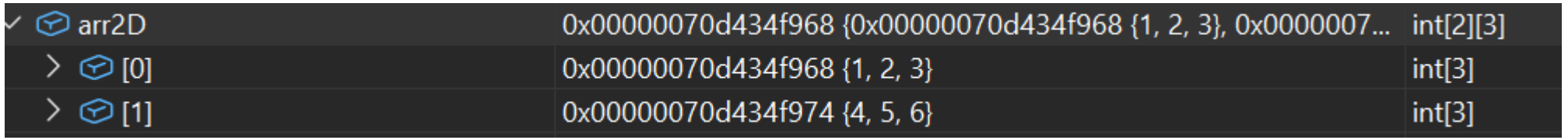
3.3一维数组

arr1D	0x000000a461bcf8c8 {1, 2, 3}	
[0]	1	
[1]	2	
[2]	3	

3.4 指向一维数组的指针变量(如何查看地址、值?

3.5二维数组(包括数组名仅带一个下标的情况)

```
int arr2D[2][3] = { {1, 2, 3}, {4, 5, 6} }; // 二维数组
```



3.6 实参是一维数组名，形参是指针的情况，如何在函数中查看实参数组的地址、值?

```
int arr1D[3] = { 1, 2, 3 }; // 一维数组
arrayFunc(arr1D, 3);        // 自定义函数（知识点1.5/1.6/3.6）
```

```
14  void arrayFunc(int* arr, int len) {  // 形参为指针（知识点3.6）
15      for (int i = 0; i < len; i++) {
16          arr[i] = arr[i] * 2;
17      }
```

arr	0x0000006452affc98 {1}	int *
1		int

3.7 指向字符串常量的指针变量(能否看到无名字符串常量的地址?

3.8 引用(引用与指针是否有区别?有什么区别?)

不能看到地址

```
const char* str = "hello"; // 指向字符串常量的指针
int& ref = a;              // 引用
```

a	10	int
ref	10	int &

引用无独立地址，别名；指针独立地址

3.9 使用指针时出现了越界访问

```
int bad[2] = { 0, 0 };
bad[5] = 999;  // 越界
```

```
1>D:\learn\高程作业\vs2026test\vs2026test\test.cpp(38,1): error C4789: 函数“main”中的缓冲区“bad”(大小为 8 字节)将溢出；从偏移 4 时开始，将会写入 20 字节
```